Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025 - 26

Course: Machine Learning Lab

Course code: CSL604

Year: TE3 SEM: VI

Experiment Number: 07
Aim:To implement Single Layer Perceptron Learning algorithm.
Name: Satyam Maurya
Roll Number: 62
Date of Performance:09-03-26
Date of Submission: 16-03-26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal Work & Timely Submission	10	
Total	20	

Performance Indicator	Expected Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal Work & Timely Submission	10	6	4

Checked by:

Name of Faculty: Ms. Nivedha Raut

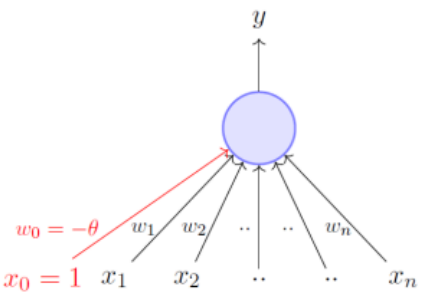
Signature:

Date:

Aim: To implement Single Layer Perceptron Learning algorithm

Description:

• A Perceptron developed by Frank Rosenblatt(1968) is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It is mainly used for binary classification problems.
• Takes multiple inputs and assigns weights
• Computes a weighted sum and applies a threshold
• Outputs either 0 or 1 (binary outcome)
• It forms foundation of larger neural networks
• It is combination of MP model + Hebb learning rule
• The perceptron network consists of three units, namely, sensory unit (input unit) and response unit (output unit).
• The perceptron learning rule is used in the weight updating between the associator unit and the response unit.
• For each training input, loss is calculated as difference between Prediction value and actual value.



$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i \geq \theta$$
$$= 0 \quad if \sum_{i=1}^{n} w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i - \theta \geq 0$$
$$= 0 \quad if \sum_{i=1}^{n} w_i * x_i - \theta < 0$$

A more accepted convention,

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$
$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

$$where, \quad x_0 = 1 \quad and \quad w_0 = -\theta$$

- PERCEPTRON LEARNING

New wi = Old wi + Δwi
Δwi = ηtxi (text book) or η(t-o)xi used by sciket library (Python Implementation) Where
O= Observed value/Prediction
t = is the target/actual value,
η Is a small constant called learning rate.
➢ If the output is correct the weights wi are not changed
➢ If the output is incorrect the weights wi are changed such that the output of the perceptron for the new weights is closer to t.
➢ The algorithm converges when Δwi → 0 and
• Training data is linearly separable
• η is sufficiently small

Code:

```
import numpy as np

# Activation function (Step function)
def step_function(x):
    return 1 if x >= 0 else 0

# Perceptron Training Function
def train_perceptron(X, y, learning_rate, epochs):
    n_features = X.shape[1]

    # Initialize weights and bias
    weights = np.zeros(n_features)
    bias = 0

    print("\nInitial Weights:", weights)
    print("Initial Bias:", bias)

    # Training loop
    for epoch in range(epochs):
        print(f"\nEpoch {epoch+1}")
        for i in range(len(X)):
            linear_output = np.dot(X[i], weights) + bias
            y_pred = step_function(linear_output)

            error = y[i] - y_pred

            # Update weights and bias
            weights += learning_rate * error * X[i]
```

```
        bias += learning_rate * error

        print(f"Input: {X[i]}, Target: {y[i]}, Predicted: {y_pred}, Error: {error}")

    return weights, bias

# Prediction function
def predict(X, weights, bias):
    outputs = []
    for x in X:
        linear_output = np.dot(x, weights) + bias
        outputs.append(step_function(linear_output))
    return outputs


n = int(input("Enter number of training samples: "))
features = int(input("Enter number of features: "))

X = []
y = []

print("\nEnter input values:")
for i in range(n):
    row = list(map(float, input(f"Input {i+1}: ").split()))
    X.append(row)

print("\nEnter target outputs (0 or 1):")
for i in range(n):
    target = int(input(f"Output {i+1}: "))
    y.append(target)

X = np.array(X)
y = np.array(y)

learning_rate = float(input("\nEnter learning rate: "))
epochs = int(input("Enter number of epochs: "))

# Train model
weights, bias = train_perceptron(X, y, learning_rate, epochs)

print("\nFinal Weights:", weights)
print("Final Bias:", bias)
```

```
# Testing
print("\nPredictions:")
predictions = predict(X, weights, bias)
for i in range(len(X)):
    print(f"Input: {X[i]} -> Predicted Output: {predictions[i]}")
```

Output:

```
Enter number of training samples: 4
Enter number of features: 2

Input 1: 0 0
Input 2: 0 1
Input 3: 1 0
Input 4: 1 1

Output 1: 0
Output 2: 1
Output 3: 1
Output 4: 1

Learning rate: 0.1
Epochs: 5
```

Conclusion:

The Single Layer Perceptron Learning Algorithm is a simple yet powerful supervised learning technique used for binary classification problems. In this experiment, the perceptron successfully learned the relationship between input and output by iteratively adjusting weights and bias based on error correction.

It was observed that the perceptron performs well for linearly separable problems such as AND and OR gates, where it correctly classifies all inputs after sufficient training. However, it fails to solve non-linearly separable problems like XOR, which highlights its limitation.

Overall, the perceptron forms the foundation of neural networks and helps in understanding how machines can learn from data using basic mathematical concepts.